

Remember: Beginning is the hardest part.

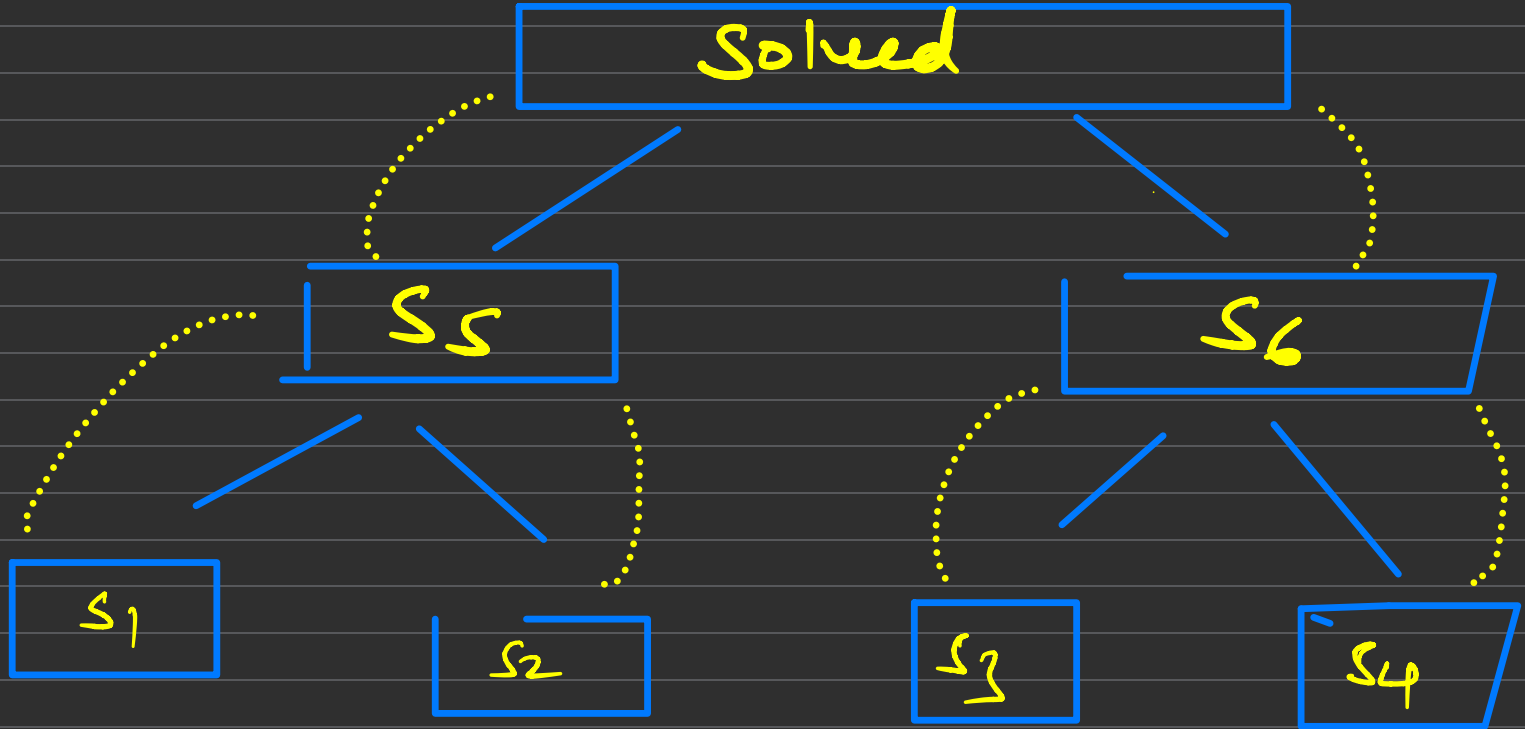
A photograph of a desk setup. On the left, a stack of books in various colors (red, blue, orange, black) is next to a light-colored, textured ceramic vase. In the center, a silver laptop is open, displaying a large line-art icon of a medal with a checkmark inside. To the right of the laptop, a pair of glasses with thin frames and a black cup are visible. The background is a plain wall.

DIVIDE & CONQUER

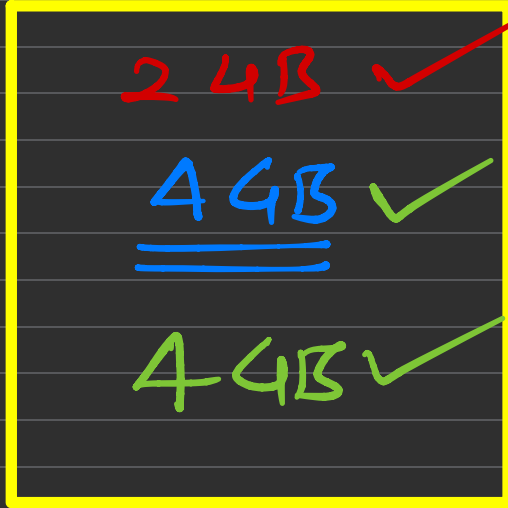


QS & MS

What is Divide and Conquer (Technique)



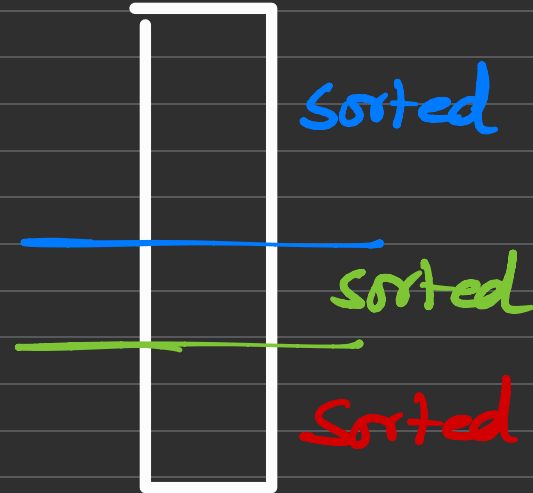
4 GB



RAM

(Sort)

10 GB



DATA

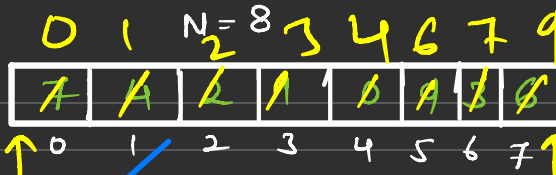


Merge Sort

$N = 8$

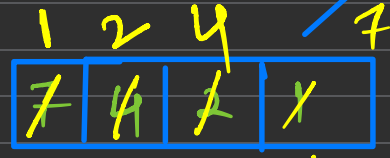
7	4	2	1	0	9	3	6
0	1	2	3	4	5	6	7

— Works on principle of DnC



(start + end)
2

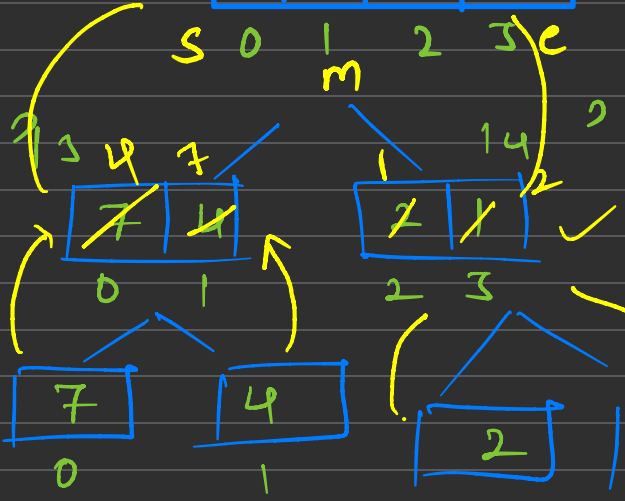
P₁



P₂

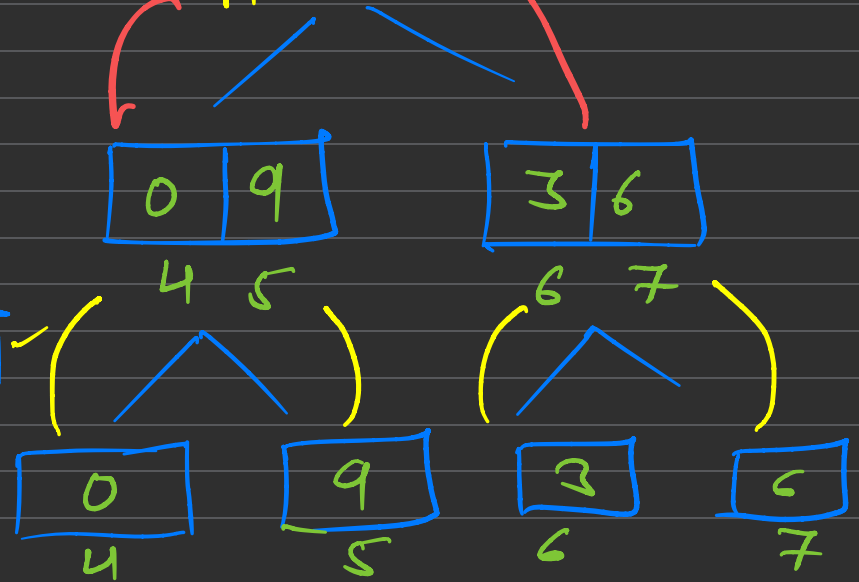


(Waiting)



(s) 4 5 6 7 (e)

m



N = 8

7	4	2	1	0	9	3	6
0	1	2	3	4	5	6	7

```
void mergeSort(arr, s, e)
{
```

```
    if (s == e) return
```

```
    mid = (s + e) / 2
```

```
    mergeSort(arr, s, m)
```

```
    mergeSort(arr, m + 1, e)
```

```
    merge(arr, s, m, e)
```

```
}
```

```
void merge(arr, s, m, e)
{
```

low = start (P₁)

high = mid + 1 (P₂)

vector<int> temp

```
while (low ≤ mid &&
       high ≤ end)
{
```

```
if (a[low] ≤ a[high])
{
    temp.push(a[low])
    low++
}
```

N = 8

7	4	2	1	0	9	3	6
0	1	2	3	4	5	6	7

(s)	(m)	(e)					
7	4	2	1	0	9	3	6
0	1	2	3	4	5	6	7
(low)				(high)			

Prob₁ = start → mid

Prob₂ = mid + 1 → end

				h		h	
-				→			
7	4	2	1		0	9	3
0	1	2	3		4	5	6

else {

temp.push(a[high])

high++

}

}

0	7	4	2	1
---	---	---	---	---

while (low ≤ mid)

{ temp.push(a[low])

low++

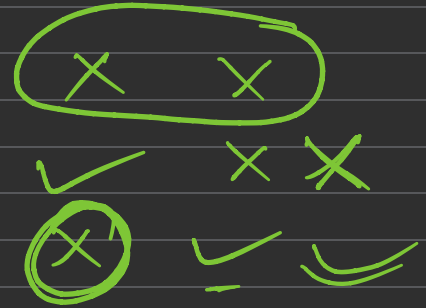
}

while (high ≤ end)

{ temp.push(a[high])

high++

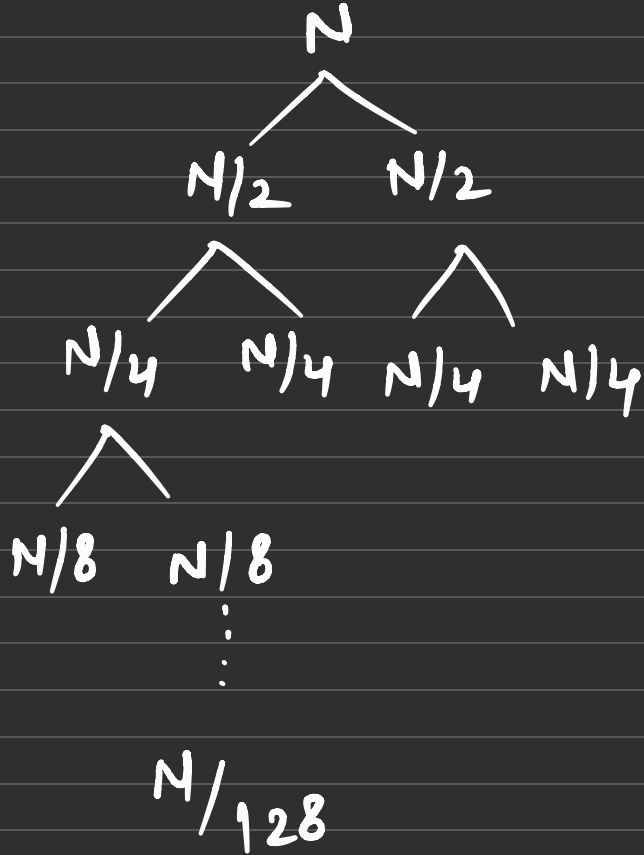
}



for ($i = s$; $i \leq e$; $i++$)

arr[i] = temp[i - start]

└



$$\frac{N}{2^K} = 1$$

$$N = 2^K$$

$$\log N = \log 2^K$$

$$\log N = K \log 2$$

$$\frac{\log N}{\log 2} = K = \boxed{K = \log_2 N}$$

To combine = $O(N)$

T.C = $O(N \log N)$

~~N ways~~

(s) (m)

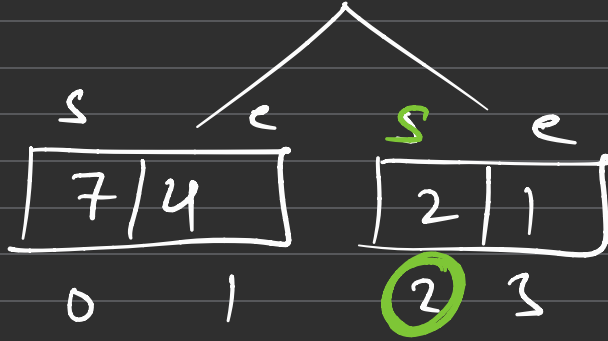
7	4	2	1
0	1	2	3

(low)

(e)

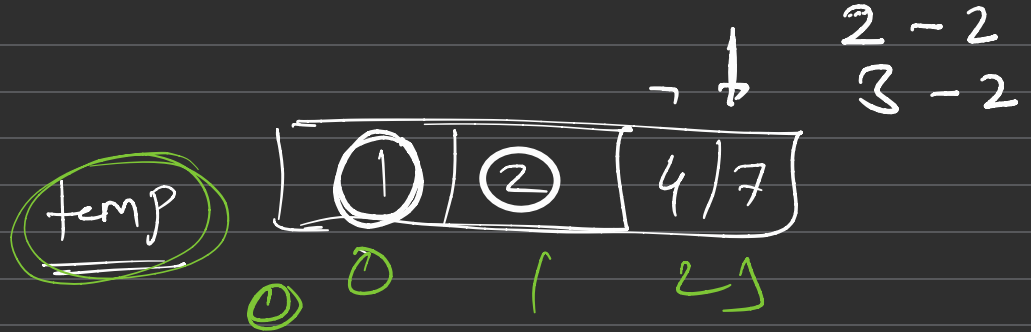
0	9	3	6
4	5	6	7

(high)



for ($i = s$: $i \leq e$: $i++$)

$a[i] = temp[i - s]$

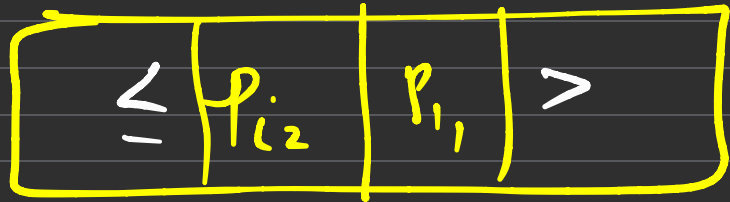


Quick Sort

$N = 8$

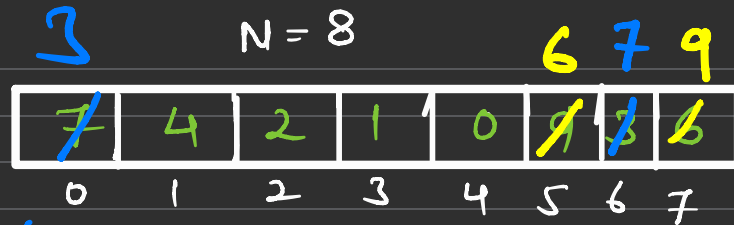
7	4	2	1	0	9	3	6
0	1	2	3	4	5	6	7

- pivot Element left right



pivotIndex = 3

pivot = q[s](7)



if cross(s, e)

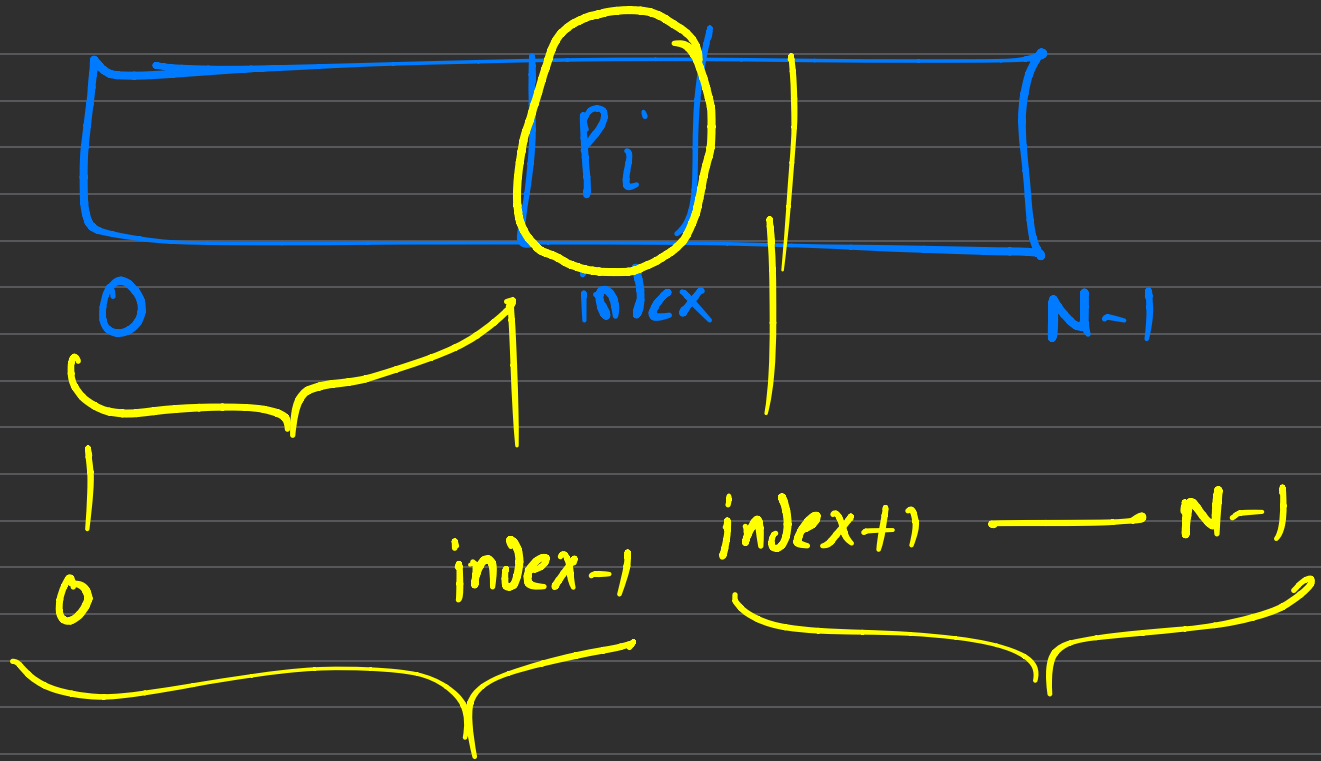
swap(pi, end)

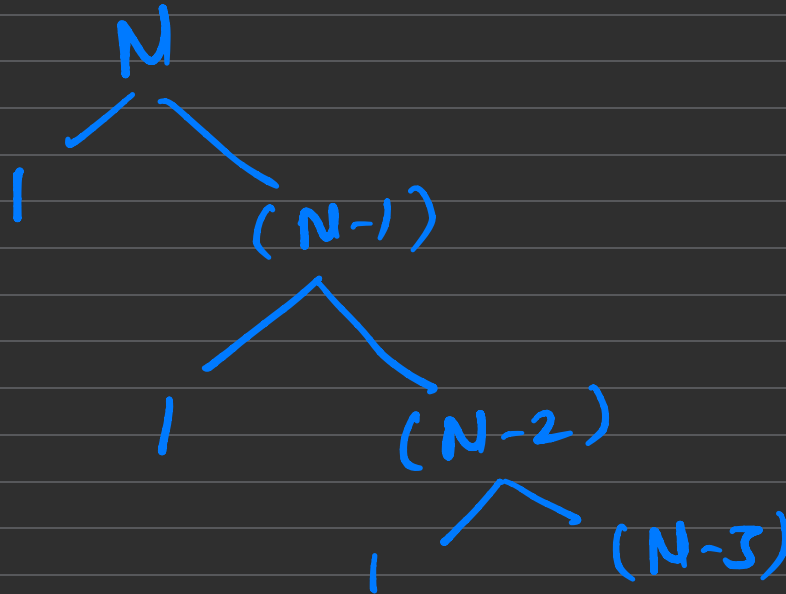
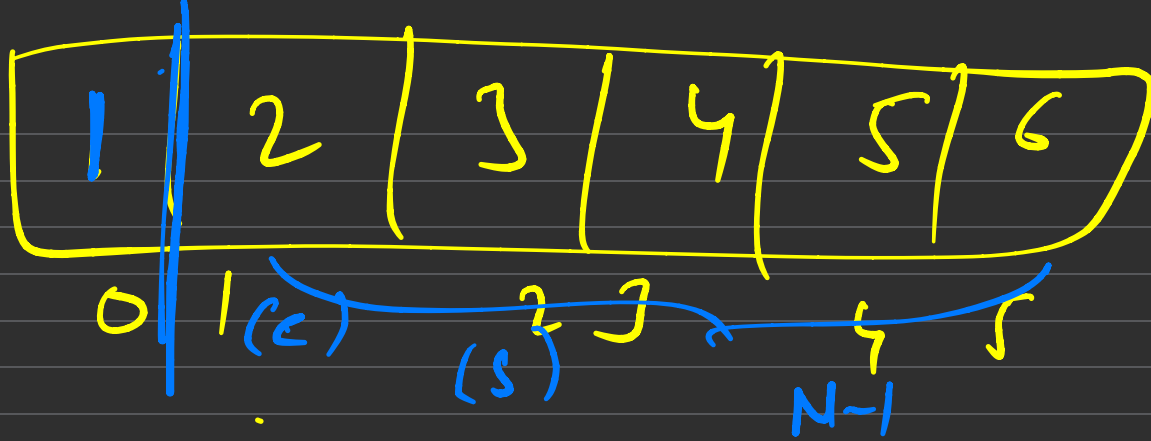
return end

if (q[s] ≤ pivot)

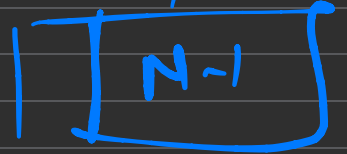
^{s++}
if (q[e] > pivot)

else ^{c--}
swap(s, e)





pivot = 1/2



$$(N + N-1 + N-2 + N-3)$$

$$9 + 8 + 7 + 6 + 5 + 4 + 3 + \dots$$

$$1 \longrightarrow N$$

$$\frac{N \times (N+1)}{2}$$



if already
Sorted

$$\frac{N^2 + N}{2} = \boxed{O(N^2)}$$

$O(N \log N)$

Not Sorted

